# Hazard Analysis
# Natural Control

**Group 7**
Lucas DeBoer
Kevin Kim
Aaron Loh
Curtis Vos

Table 1: Revision History

| Date | Developer(s) | Change |
|---|---|---|
| 2026-10-14 | All | First draft from team discussion |
| 2026-10-16 | Lucas DeBoer | Revise initial draft |
| 2026-10-17 | Aaron Loh | Align FMEA and safety requirements with revised components |
| 2026-10-17 | Aaron Loh | Add interview findings to assumptions and H2-a |
| 2026-10-17 | Aaron Loh | Add consent and credential vault to H5-a and H7-c |

[TODO: update this.]

# Contents


**1 Introduction** **1**

**2 Scope and Purpose of Hazard Analysis** **1**

**3 System Boundaries and Components** **1**

**4 Critical Assumptions** **2**

**5 Failure Mode and Effect Analysis** **3**

**6 Safety and Security Requirements** **8**
6.1 Functional Safety Requirements . . . . . 8
6.2 Non-Functional Safety Requirements . . . . . 9

**7 Roadmap** **10**

**8 Appendix: Generative AI Use Disclosure** **11**
8.1 AI Tools Used . . . . . 11
8.2 How and Where Used . . . . . 11
8.3 Verification of AI-Generated Content . . . . . 11

**9 Appendix: Reflection** **12**

# 1 Introduction

The purpose of this project is to create a voice-based interface for a computer-use agent that aims to help visually impaired users operate their computer while receiving accessible feedback and retaining control. For the purpose of this project, a hazard is defined as any property, condition, or event that could lead to harm or loss. Physical harm is not expected, so harm is defined as a loss of the user's intended outcome, control, awareness, privacy, or account security.

# 2 Scope and Purpose of Hazard Analysis

The purpose of this document is to identify and evaluate the hazards that may arise from the use of the system by considering the potential failure modes and their effects and causes, and then mitigating these risks through relevant software requirements. The hazards below could lead to these losses:

- **Unwanted action:** Something could be sent, submitted, purchased, created, or deleted that the user did not intend. This may be costly, impossible to undo, or affect other people.
- **Missed or failed task:** An action that the user believes happened does not happen, or happens incorrectly.
- **Loss of control or awareness:** The user cannot tell what the agent is doing, cannot stop it, or is told something false.
- **Privacy leak:** Private information or credentials could be exposed.
- **Loss of independence:** If the tool slows down, stalls, or stops, the user may no longer be able to operate their computer because they may need sighted help.

These losses are amplified as a sighted user can glance at the screen to catch a mistake; a visually impaired user often cannot, so detection has to be built into the system.

# 3 System Boundaries and Components

The list below outlines the major components of the system and their roles. This is a high-level view of the system that is used to brainstorm potential hazards and is used to complete the Failure Mode and Effects Analysis (FMEA) in Section 5.

1. **Accessible client:** The layer that supports app functions outside voice control and task execution, such as start and stop keyboard shortcuts and viewing task history.

2. **Voice interface:** Captures and transcribes user voice input, and provides text-to-speech audio feedback.

3. **Task manager:** Starts, stops, and tracks tasks.

4. **AI agent:** Plans tasks and calls tools.

5. **Action gate:** Handles approval of consequential actions.

6. **Outcome verification:** Verifies completed actions match what the user asked for.

7. **Action tools and integrations:** Tools the agent uses to gather information and perform actions.

8. **Task history layer:** Stores pending approvals, action history, task state, and any other relevant information needed to support recovery after disconnects.

The following are outside the system boundary: the user, laptop hardware (including the microphone and speakers), the operating system (macOS), the network, AI and speech service providers, websites, and external accounts and services. Failures of these are not analyzed internally, but the system's response to them is. [TODO: Add the component diagram (?).]

# 4 Critical Assumptions

1. Setup done correctly.
   A setup helper completed the assisted setup (accounts connected, permissions granted), so setup errors are not analyzed as runtime hazards. Setup help may only be available weekly (for example, from an intervenor), so recovery after setup should not depend on it.

2. One authorized speaker.
   There is only one user giving commands; other voices are noise. We do not need to verify who is speaking.

3. User's own accounts.
   Connected accounts belong to the user, who may act on them. Development and evaluation use test accounts.

4. Good faith user.
   The user does not purposely misuse the system or cause harm or loss. The user also does not intentionally give misleading or vague commands; they always strive to give instructions to the best of their ability.

5. Compatible device.
   The user has a functioning laptop with a microphone and speakers running macOS with Chrome installed. Note that we will consider scenarios where

the laptop has a microphone and speakers that fail mid-usage, but all laptops are assumed to have these components.

6. Users can understand spoken output.
   We assume that the user can understand the spoken output of the system. We do not analyze scenarios where the user has both vision and hearing loss. This excludes many deafblind users, such as the clients of CNIB Deafblind Community Services (from our October 17 interview [TODO: link to interview?]); braille output through VoiceOver is a possible future extension.

We do not assume that the network, speech recognition, the AI model, websites, or other external services work correctly. Their failures are analyzed in the FMEA.

# 5 Failure Mode and Effect Analysis

The following table lists potential failure modes of each component, their effects and causes, recommended actions to mitigate the hazards, and the associated safety requirements. Each row is one failure mode of one component. Hazard IDs are `H<component>-<letter>`. How a failure is detected without sight is part of the recommended action, since a blind user often cannot notice a failure visually.

Table 2: Failure Mode and Effect Analysis

| Ref | Component | Failure mode | Effects | Causes | Recommended action | SR |
|---|---|---|---|---|---|---|
| H1-a | Accessible client | Stop control can't be reached without voice | User can't halt the agent if voice fails; loss of control | Shortcut clashes with VoiceOver; focus in another app; client frozen | Provide a stop shortcut that works whenever the client runs and doesn't clash with VoiceOver; test with VoiceOver on | FR.Sa.1 |
| H2-a | Voice interface | Misunderstands the user: wrong words, names, numbers, a cut-off sentence, or another voice | Wrong task or wrong details acted on | Background noise, accents, turn detection ending too early, TV or other people | Read back key details before every consequential action; let the user correct one part of the read-back without restarting | FR.Sa.3 |
| H2-b | Voice interface | A stop request isn't recognized | Task keeps running; loss of control | Stop request classified as chatter; mistranscription; user talking over the system | Recognize stop phrases with a fixed rule before any model interprets them; keyboard stop as backup; acknowledge quickly | FR.Sa.4, NFR.Sa.1 |
| H2-c | Voice interface | Cannot receive or transcribe voice input or give voice output | User can't give commands, approvals, or stop, or can't hear feedback while the agent is working; loss of control and awareness | Transcription or speech service failure; input or output device disconnected or removed | Detect loss of input or output where possible; stop the current task and allow no further requests until everything is functioning | FR.Sa.20, FR.Sa.5 |
| H3-a | Task manager | Stop is acknowledged but new work starts | Unwanted actions after stop; false sense of control | Interruption occurs only at tool boundaries; an operation is already submitted | Treat "stop working" as pause. Block new task actions and preserve context. Report acknowledgment and confirmed pause separately. Explain what completed and what remains uncertain. Resume only on a clear user request. "Cancel" ends the task. Earlier actions are not undone. | FR.Sa.5 |

| Ref | Component | Failure mode | Effects | Causes | Recommended action | SR |
|---|---|---|---|---|---|---|
| H3-b | Task manager | A consequential action is repeated after reconnecting | Duplicate task | Automatic retry after a dropped connection when the first attempt actually succeeded | Unique ID per approved action; never retry a consequential action automatically; pause new actions while disconnected; on reconnect, report pending items | FR.Sa.6, FR.Sa.7, NFR.Sa.2 |
| H4-a | AI agent | Does the wrong task or picks the wrong target | Wrong action prepared (wrong thread, wrong date) | Ambiguous request; model error | Ask a clarifying question when required details are missing or ambiguous; the approval read-back (H2-a) catches the rest | FR.Sa.9 |
| H4-b | AI agent | Claims outcome that didn't happen | Missed task; duplicated task; user misinformed | Model hallucination; partial failure reported as success; partial success reported as failure | Report completion only from verified outcomes; distinguish verified, partial, and unverified results | FR.Sa.10 |
| H4-c | AI agent | Stalls or loops | Silence or repeated steps; user unsure what's happening; cost grows | Model loop; slow or broken website | Speak a status update at a regular interval while a task runs; after a step or time limit, pause and ask the user | FR.Sa.11, NFR.Sa.3 |
| H4-d | AI agent | AI provider outage or timeout | Task halts partway | Provider down; network loss | Pause and tell the user what was and wasn't done; never claim success | FR.Sa.12 |
| H5-a | Action gate | A consequential action slips through without approval | Unwanted action | Action not recognized as consequential (e.g. a browser click that submits a form); new tool not classified | Default-deny: anything not known to be read-only requires approval | FR.Sa.13 |
| H5-b | Action gate | A stale approval is reused | Action runs with details the user never approved | Details changed after approval (time, recipient) | Bind each approval to its exact details; approvals are single-use; any change requires new approval | FR.Sa.14 |
| H5-c | Action gate | An approval request expires or is withdrawn without a clear result | User unsure whether the action happened | Timeout; disconnect; request superseded | Tell the user explicitly that the action did not happen and what is still pending | FR.Sa.15 |

| Ref | Component | Failure mode | Effects | Causes | Recommended action | SR |
|---|---|---|---|---|---|---|
| H5-d | Action gate | A delete is approved by mistake | Item or data lost, potentially permanently | Misheard approval; wrong item matched; user assumes the action can be undone | For deletes, read back identifying details and say it cannot be undone; use recoverable deletion (trash) where the service supports it | FR.Sa.19 |
| H6-a | Outcome verification | Verification wrongly confirms success, or cannot check and reports success anyway | User believes a task succeeded when it did not; missed task | Checking the wrong item or a stale copy of the page or account; the service does not expose the result | Re-read the specific item the action changed after acting; if it cannot be checked, report the result as unverified | FR.Sa.10 |
| H6-b | Outcome verification | Verification wrongly reports failure, or cannot check and reports failure anyway | User retries, causing a duplicate action; user loses trust in verification | Checking a stale copy or the wrong item; the service updates slowly after the action | Before reporting failure re-check the item; if the result is still uncertain, report it as unverified, not failed | FR.Sa.10 |
| H7-a | Action tools and integrations | Acts on the wrong page element | Wrong item chosen or wrong field filled | Poor labels; unusual layout; page changed after reading | Re-check the target against the approved details immediately before acting; verify the result afterward | FR.Sa.10, FR.Sa.14 |
| H7-b | Action tools and integrations | Follows instructions injected by a webpage | Unwanted action or private data sent elsewhere | Page or email text written to manipulate agents | Treat page and email content as data, never as commands; consequential actions must trace to the user's request; tell the user when content tries to give instructions | FR.Sa.16 |
| H7-c | Action tools and integrations | Access is broader than needed | Any other failure can do more damage | Requesting broad OAuth permissions for convenience | Request the narrowest permissions each tool needs; keep tokens out of the repository and logs; store and use passwords securely | FR.Sa.17 |
| H7-d | Action tools and integrations | Outcome unknown after a timeout | Retrying risks a duplicate; not retrying risks a missed task | Network or API timeout | Check the service or item state directly and get user approval before any retry | FR.Sa.18 |

| Ref | Component | Failure mode | Effects | Causes | Recommended action | SR |
|---|---|---|---|---|---|---|
| H8-a | Task history layer | Task history doesn't match what happened | User can't review or catch wrong actions afterward | History written from the agent's claims instead of recorded events; events lost during disconnects | Build history only from recorded actions and verified outcomes; label anything unverified | FR.Sa.2 |
| H8-b | Task history layer | Task state or pending approvals lost | User never asked, or told the wrong status | Crash or restart of the task manager or agent | Save task state and pending approvals; after a restart, announce interrupted tasks and their last known status | FR.Sa.8 |

# 6 Safety and Security Requirements

These requirements come from the recommended actions in Table 2 and are also part of the SRS. FR.Sa marks functional safety requirements and NFR.Sa non-functional ones. Timing values are named constants, set after POC measurements (see the SRS constants table [TODO: link to srs constant table ?]).

## 6.1 Functional Safety Requirements

- **FR.Sa.1:** The system shall provide a keyboard stop control that works whenever the client is running and does not conflict with VoiceOver commands. (H1-a)
- **FR.Sa.2:** The system shall build the task history only from recorded actions and verified outcomes, labelling unverified entries. (H8-a)
- **FR.Sa.3:** Before each consequential action, the system shall read back its key details, allow users to change parts of the request, and proceed only after explicit approval. (H2-a)
- **FR.Sa.4:** The system shall recognize stop requests with a fixed rule that does not depend on model interpretation. (H2-b)
- **FR.Sa.5:** "Stop working" shall pause the task and block new task actions. The system shall preserve context and distinguish acknowledgment from confirmed pause. Already-submitted operations may still finish. The system shall report completed work, unfinished work, and uncertain outcomes. Only checks of earlier outcomes may continue while paused. Work shall resume only on a clear user request. "Cancel" shall end the task without undoing earlier actions. (H2-c, H3-a)
- **FR.Sa.6:** The system shall assign each approved consequential action a unique ID and shall not repeat an action with the same ID. (H3-b)
- **FR.Sa.7:** The system shall pause the task while the user is disconnected. (H3-b)
- **FR.Sa.8:** The system shall save task state and pending approvals, and after a restart shall announce interrupted tasks with their last known status. (H8-b)
- **FR.Sa.9:** The system shall ask a clarifying question when details required for an action are missing or ambiguous. (H4-a)
- **FR.Sa.10:** The system shall report a task as a success or failure only after verifying its outcome, and otherwise shall report it as partial or unverified. (H4-b, H6-a, H6-b, H7-a)
- **FR.Sa.11:** The system shall pause a task and ask the user after a configured step or time limit is reached. (H4-c)

- **FR.Sa.12:** If an external service fails mid-task, the system shall pause and tell the user what was and was not completed. (H4-d)
- **FR.Sa.13:** The system shall require approval for any action not classified as read-only. (H5-a)
- **FR.Sa.14:** The system shall bind each approval to the action's exact details, use it once, and require new approval if any detail changes. (H5-b, H7-a)
- **FR.Sa.15:** When an approval request expires or is withdrawn, the system shall tell the user that the action did not happen. (H5-c)
- **FR.Sa.16:** The system shall treat webpage and email content as data, and shall only perform consequential actions that trace to the user's request. (H7-b)
- **FR.Sa.17:** The system shall request only the account permissions each tool needs, and shall store and use credentials securely, keeping them out of source control, logs, and the AI model input. (H7-c)
- **FR.Sa.18:** The system shall check the service or item state directly and confirm the user's approval before retrying an action whose outcome is unknown. (H7-d)
- **FR.Sa.19:** For delete actions, the system shall read back the item's identifying details, state that the action cannot be undone, and use recoverable deletion (such as moving to trash) where the service supports it. (H5-d)
- **FR.Sa.20:** If voice input, voice output, or speech services are unavailable, the system shall immediately stop the current task, take no further actions until voice is restored, and notify the user when the system is functional again. (H2-c)

## 6.2 Non-Functional Safety Requirements

- **NFR.Sa.1:** The system shall audibly acknowledge a stop request within STOP_ACK_TIME of the end of the utterance or key press. (H2-b)
- **NFR.Sa.2:** The system shall notify the user of a lost connection within CONNECTION_NOTICE_TIME, by sound if speech is unavailable. (H3-b)
- **NFR.Sa.3:** While a task is running, the system shall give a spoken status update after IDLE_PROMPT_TIME without spoken progress. (H4-c)

# 7 Roadmap

**POC (November).** FR.Sa.3, 4, 5, 13, 14; NFR.Sa.1. The safeguards the demo is meant to prove. FR.Sa.13 includes testing whether browser actions can be intercepted.

**Before Rev 0 / participant evaluation.** FR.Sa.1, 2, 6, 7, 8, 9, 10, 12, 15, 16, 17, 18, 19, 20; NFR.Sa.2. These matter as soon as real participants use real-looking accounts.

**Later.** FR.Sa.11; NFR.Sa.3. Basic status updates exist earlier, but tuned step/time limits and update frequency need usability evidence. (Team to confirm.)

# 8 Appendix: Generative AI Use Disclosure

## 8.1 AI Tools Used

Aaron Loh: Claude Code with Claude Opus 5.5.

OpenAI Codex helped revise H3-a and FR.Sa.5 from Aaron's decisions and Kevin's review.

[TODO: Other members: add the tools you used.]

## 8.2 How and Where Used

Aaron used Claude to work through this analysis. It asked questions about scope, losses, components, and failure modes, summarized the Hermes, LiveKit, and Browser Use documentation, and translated the decisions into written drafts of each section. Aaron made and reviewed every decision.

## 8.3 Verification of AI-Generated Content

This is a first draft. Before submission the team will review every hazard and requirement, check the cited documentation, rewrite sections in our own words, and keep this disclosure up to date. The team is responsible for the submitted document.

# 9 Appendix: Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?
4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?